

METHODS AND EXCEPTION HANDLING

CONTENTS OF THE CHAPTER

- *The concepts of methods, static methods, and static variables.*
- *Parameters and parameter types.*
- *Method overloading*
- *Exceptions and exception handling.*
- *Create documentation using Javadocs.*

Chapter 4 focuses on methods, particularly static methods. Static methods allow Java to write code similar to procedural programming. Key concepts in this section include distinguishing between static and instance methods, static variables, types of declared and passed parameters, and creating and using methods. Additionally, readers will learn about the usefulness and rules of using dynamic parameters, array parameters, and method overloading. Exception handling is also a crucial topic in this chapter. The aim is to guide learners on different types of exceptions, how to create them, and how to throw them. The chapter concludes with a section on creating documentation using Javadocs, a tool that allows programmers to quickly and correctly create project documentation using Java documentation guidelines.

4.1. METHOD

4.1.1. The concept of method

A **method** is a set of statements that perform certain tasks defined in a class, grouped together and given a name. A method can be understood as a **"black box"** :

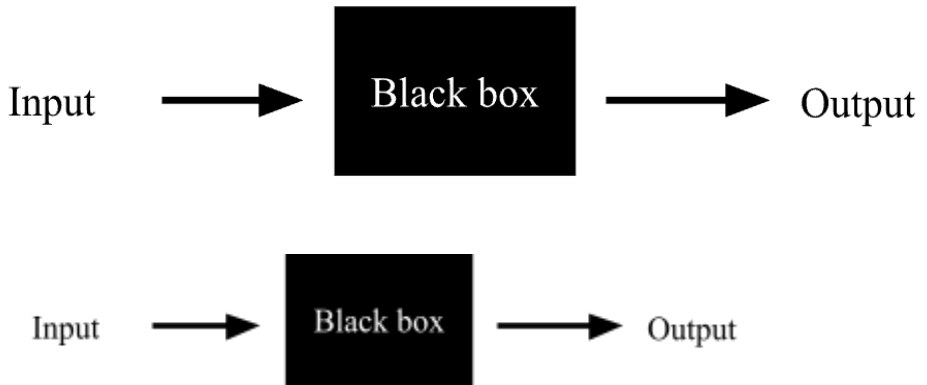


Figure 4-1. Methods can be understood as black boxes.

Methods allow a potentially very complex task to be solved as a **single concept**. Instead of worrying about every step the computer might have to take to perform that task, just remember the method's name. Whenever you want the program to perform the task, just call the method. Methods are essentially a block of statements to perform a certain job. A method is often compared to a **"black box"** because it's impossible to see what's "inside" it (or more accurately, you don't want to see inside it, because then you'd have to face the complexity of the statements inside each method). A black box needs information to tell how the external environment can interact with it. A television can be considered a black box; the control buttons on the television are the way the external environment interacts with this black box.

4.1.2. Black Box rules

The rules regarding the black box are also the rules that will apply to the method:

The interface should be simple, clear, and easy to understand : In the real world, people use black boxes a lot. Televisions, cars, mobile phones, refrigerators can all be considered black boxes... For televisions, you can turn them on and off, change channels, and adjust the volume using components of the TV interface such as the power switch and remote control. The same applies to mobile phones, although the interface in that case is much more complex. This rule shows the need to design a method

that is easy to use and easy to access by other methods.

To use it, you don't need to understand how it works, just know the user interface: In fact, it's possible to change the implementation inside the black box, but the external appearance remains unchanged. For example, when the TV switches from one technology to another, the TV user doesn't need to know about it – or even what it means. Similarly, it's possible to rewrite the inside of a method, for example, using more efficient code without affecting programs that use that method.

When creating a black box, you don't need to know too much about the systems it will be used in: Of course, to have a black box, someone has to design and build the implementation from scratch. Then, the black box can be used in countless different scenarios. The person implementing what works inside the black box doesn't need to know too much about the larger system it will be used in. The implementer simply needs to ensure that the black box performs its assigned task and interfaces correctly with the rest of the world.

4.2. DEFINITION OF METHOD

A method **must** be defined in a class using the following syntax:

```
modifiers return-type function-name(parameter-list) {  
    statements;  
    return;  
}
```

Defining a method should include:

- **function-name (the method name)** contains enough information to call the method, and the code will be executed each time the method is called.
- **Statements enclosed in curly braces {and} in a method definition form the body of the method.** These statements are the inner workings or implementation of the “black box,” as discussed in the previous section. These are the statements that the computer executes when the method is called .

- **modifier** that may appear at the beginning of a method definition are words that establish certain characteristics of the method.
- **return-type** keyword is used to specify the type of value to be returned if a method's job is to compute some value. The method performs the computation using statements and returns a value using the `return` keyword. The return value can be a String or an integer, or even an array type like `double[]`. If the method does not return a value, the return type is replaced with the special `void`, indicating that no value is returned. The term "void" indicates that the return value is empty or does not exist.
- A **parameter list** is part of a method's interface. It represents information passed into the method from the outside. This information is then used by the method's internal calculations. A method's parameter list can be empty, or if it contains multiple parameters, they will be separated by commas.

Below are a few examples of method definitions:

```
public static void playGame() {
    // "public" and "static" are modifiers that identify this as
    // a public static method;
    // "void" is the return type;
    // "playGame" is the method name;
    // empty parameter list.
    // These statements define what playGame does here.
}

int getNextN (int N) {
    // No complements;
    // "int" is the return type;
    // "getNextN" is the method name;
    // The parameter list includes a parameter named "N" of type
    // "int".
    // The statements defining getNextN are executed here.
}

static boolean lessThan (double x, double y) {
```

```
// "static" is a modifier that specifies this is a static
method;
// "boolean" is the return type;
// "lessThan" is the method name;
// The parameter list includes two parameters named "x" and
"y", and the type of each parameter is "double".
// These statements define what lessThan does here.
}
```

In the second example, `getNextN` is a **non-static** method because its definition does not include the keyword “**static**”. Another modifier shown in the examples is “**public**”. This modifier indicates that the method can be called from anywhere in the program, even from outside the class where the method is defined. There is another modifier, “**private**”, which indicates that the method can only be called from methods within the same class. If no access specification is given for a method, its access scope is called the **default**, meaning that the method can be called from anywhere in the package containing that class, but not from outside the package.

Note that `main()` is also a typical example of the method:

```
public static void main (String [] args) {

}
```

In the above method, the modifiers are public and static, the return type is void, the method name is main, and the parameter list is “String[] args”. In this case, the type for the parameters is array type String[].

4.3. STATIC METHOD

4.3.1. The concept of static methods

A static method is a method defined with the ‘**static**’ keyword. A static method belongs to a class and is called through the class, not through an object of the class. This means that a static method will be called without creating an instance of the class. Static methods can access static variables and can change their values.

A static method exists independently of any object and is stored in memory as part of the class's method area. It is loaded into memory when the class is loaded by the JVM, which typically happens the first time the class is referenced. Because of this, a static method has a single, shared existence throughout the entire lifetime of the application, regardless of how many objects of the class are created—or even if no objects are created at all. This explains why a static method can be invoked using the class name and why it cannot directly access instance members: instance variables and instance methods only exist in memory after an object is created on the heap, while the static method already exists and operates without any object context.

4.3.2. Calling static methods

To **call** a static method from another static method in the class, write the method name and pass the input parameters. For example:

```
playGame();
```

To call a method from another class, you need to specify the class name:

```
Poker.playGame();
```

Method calls can occur anywhere within the same class where the method is defined. Since **playGame()** is a public method, it can also be called from other classes, but in that case, the computer must be told which class it's coming from. Because **playGame()** is a static method, its full name includes the name of the class in which it's defined. For example, suppose **playGame()** is defined in a class called **Poker**. Using the class name here tells the computer which class to look for to find the method. It also allows differentiation between **Poker.playGame()** methods and other **playGame()** methods defined in other classes, such as **Roulette.playGame()** or **Blackjack.playGame()**. Note that the parameter list can be empty, as in the **playGame()** example, but must always contain parentheses. The number and data types of the parameters provided when calling a method must match the number and data types specified in the parameter list in the

method definition.

More generally, the method call statement for a static method looks like this:

```
class_name.method_name(parameters);
```

Consider the following example:

```
public class Main {  
    static void myStaticMethod() { // Static method  
        System.out.println("Static methods can be called  
        without using an object");  
    }  
  
    void myPublicMethod() { // Non-static method  
        System.out.println("This method needs to be called  
        through an object");  
    }  
  
    public static void main(String[] args) { // Main method  
        myStaticMethod(); // Call static method  
        Main myObj = new Main(); // Create object  
        myObj.myPublicMethod(); // Call the method from the  
        object  
    }  
}
```

The results are displayed:

```
Static methods can be called without using an object.  
This method needs to be called through the object.
```

4.4. STATIC VARIABLE

Member variables, also known as attribute variables, are variables declared within a class in a similar way to a method. These variables are then referred to as attributes of the class. For example:

```
class Person{
```

```
String userName;  
}
```

A member variable can be declared as a **static variable**. When using the additional keyword **static** :

```
class Player{  
    public static int gamesPlayed;  
    public static int gamesWon;  
}
```

These static variables belong to the class and always exist even when no object is created from the variable. Memory is allocated to the variable when the class is first loaded by the Java interpreter. Any assignment statement that assigns a value to the variable will change the contents of that memory area, regardless of where the assignment statement is located in the program. Whenever the variable is used in an expression, its value will be retrieved from the same memory, regardless of where the expression is located in the program. This means that the value of a static variable can be shared by multiple methods. Because a static variable exists independently of objects, changes made to it through one reference are immediately visible to all other references, which clearly distinguishes it from instance variables that are created in the heap separately for each object.

Consider the following example:

```
package vn.edu.neu;  
  
import java.util.Scanner;  
  
public class Main {  
    static int gamesPlayed;  
    static int gamesWon;  
    public static void main(String[] args) {  
        System.out.println("Select a number from 1 to 100");  
        boolean playAgain;  
        do {  
            playGame();  
            Scanner scanner = new Scanner(System.in);
```

```

        System.out.print("Do you want to play again?");
        playAgain = scanner.nextBoolean();
    }
    while (playAgain) ;
    System.out.println("You won " + gamesWon + "/" +
gamesPlayed + " games,");
    System.out.println("Goodbye");
}

static void playGame() {
    int computersNumber;
    int usersGuess;
    int guessCount;
    computersNumber = (int) (100 * Math.random()) + 1;
    guessCount = 0;
    gamesPlayed++;
    System.out.println();
    System.out.print("What number do you guess?");
    while (true) {
        usersGuess = new Scanner(System.in).nextInt();
        guessCount++;

        if (usersGuess == computersNumber) {
            System.out.println("You guessed correctly, the
number is: " + computersNumber);
            gamesWon++;
            break;
        }
        if (guessCount == 6) {
            System.out.println("You lost, the number is " +
computersNumber);
            break;
        }
        if (usersGuess < computersNumber)
            System.out.print("The number is too low, guess
again: ");
    }
}

```

```
        else if (usersGuess > computersNumber)
            System.out.print("The number is too high, guess
again: ");
        }
    }
}
```

In the example above, the program will add two values to the static variables **gamesPlayed** and **gamesWon** to determine the number of plays and the number of wins. These two variables are used in methods such as `playGame()`, and the `main()` method will print the values of both variables when the game ends.

4.5. PARAMETER VARIABLE

If a method is a black box, **parameters** are a list of information that can be put into the black box. For example, if an air conditioner is a black box, then information such as the desired temperature and desired airflow level are called parameters. There are two types of parameters:

- **Formal parameters** (also known as **parameters**) are parameters included in a method definition.
- **actual parameters** (also known as **arguments**) is the value that will actually be passed when calling the method.

A **formal parameter** is an input variable declaration. It has a specified type such as `int`, `boolean`, `String`, or array like `double[]`. **The actual parameter** is a value that will be passed when calling/using the method. The type of the actual parameter must match the type of the formal parameter. It will be a type that allows for assignment. For example, if the formal parameter is of type `double`, it would be valid to pass an `int` as the actual parameter because an `int` can be validly assigned to a `double`.

For example, formal parameters defined in the `doTask()` method:

```
static void doTask(int N, double x, boolean test)
{

}
```

Actual parameters are the values that are input when calling a method:

```
doTask(17, Math.sqrt(z+1), z >= 10);
```

When the computer executes this command, it essentially has the same effect as a block of code:

```
{
    int N;
    double x;
    boolean test;
    N =17;
    x =Math.sqrt(z+1);
    test =(z >=10);
    // statements using the above variables
}
```

4.5.1. Flexible parameters

Variable parameters (varargs) in Java allow a method to accept a different number of parameters. For example:

```
public static int sum(int i, int... j) {
    int sum = i;
    for (int x : j) {
        sum += x;
    }
    welcome sum;
}
```

Attention:

- A method with only one varargs.
- Only the last parameter in a method can be a varargs.
- Be careful when stacking methods with varargs.

4.5.2. Array Parameters

You can specify that a **parameter's type is an array type**. For example:

```
static void printValuesInList(int[] list) {
    System.out.print('[');
    int i;
    for ( i = 0; i < list.length; i++ ) {
        if ( i > 0 )
            System.out.print(',');
        System.out.print(list[i]);
    }
    System.out.println('');
}
```

This means that an entire array can be passed to a method as a single parameter. For example, we want a method to print all the values in an array of integers, separated by commas and enclosed in square brackets. To use this method, we need an array passed to the method.

```
int[] numbers;
numbers = new int[3];
numbers[0] = 42;
numbers[1] = 17;
numbers[2] = 256;
printValuesInList(numbers);
```

The result produced by the last statement will be:

```
[42,17,256]
```

4.5.3. Command-line parameters

Command-line arguments are parameters passed when a program is called; these parameters are passed to the `main()` method. The `main()` method of a program has parameters of type `String[]`. When the `main()` method is called, some string array must be passed to `main()` as parameter values. When using the command-line interface, the user types a command to request the system to execute a program. The user can add input to this command, in addition to the program name. This input becomes a

command-line argument. The system receives the command-line arguments, places them into a string array, and passes that array to the main() method.

For example, if the program name is myProg, the user can type the following to launch the program without parameters:

```
java myProg
```

If the user wants to pass parameters, they can type:

```
java myProg one two three
```

In the above case, the command-line arguments are the strings “one”, “two”, and “three”. The system places these strings into a String[] array and passes that array as a parameter to the main() program.

For example, here's a program that prints out any command-line parameters entered by the user:

```
public class CLDemo {
    public static void main(String[] args) {
        System.out.println("You entered " + args.length + "
command-line parameter");
        if (args.length > 0) {
            System.out.println("That is :");
            int i;
            for ( i = 0; i < args.length; i++ )
                System.out.println(" " + args[i]);
        }
    }
}
```

In practice, command-line parameters are often used to pass the name of a file to a program. Nowadays, most programs run in a **graphical user interface (GUI)**, so command-line arguments are not as important as they used to be. But at least they provide an example of how array parameters can be used.

4.6. METHOD OVERLOADING

Method overloading is a capability that allows defining multiple

methods with the same name but different input parameters. For example:

```
println(int) println(double)
println(char) println(boolean)
println()
```

To call a method validly, you need to know how many formal parameters it has and the type of each parameter. This information is called **the method signature**. The signature of the `doTask()` method, used as an example above, can be represented as: `doTask(int, double, boolean)`. Note that the signature does not include the names of the parameters; in fact, if you only want to use the method, you don't even need to know the names of the formal parameters, so the parameter names are not part of the interface. Java allows different methods within the same class to have the same name, as long as **their signatures are different**. For example, the `System.out` object includes several different methods named `println`. All of these methods have different signatures, such as `println(int)` or `println(char)`. Therefore, the call `System.out.println(17)` will call the method with `println(int)`, while the call `System.out.println('A')` will call the method `println(char)`. Of course, all these different methods have the same effect.

However, it is invalid to have two methods in the same class that have the same form parameters but different return types. For example, it would be a syntax error if a class contained two methods defined as:

```
int getln() { ... }
double getln() { ... }
```

This is why the methods in the `Scanner` object are named differently, such as `nextInt()`, `nextDouble()`, etc.

4.7. EXCEPTIONS

The word "exception" is used interchangeably with "error." It encompasses any situation that arises during **program execution (runtime)**. An exception can be an error, or it can simply be a special case. Sometimes, an exception may not be considered an error, but merely another way of

organizing a program.

If an exception occurs during the execution of a program, it can be said that the exception is "thrown." When this happens, the program needs to handle this exception; otherwise, it will freeze or terminate immediately. Consider the following code example:

```
static void Main(string[] args){
    int[] mynumbers = new int[] {1,2,3};
    int i = mynumbers[9]; // an error occurs here
}
```

The above code will cause problems when run because mynumbers is an array with three elements, but if you access the 10th element using mynumber[9], an exception will be thrown. An exception will appear on the screen and the program will terminate immediately.

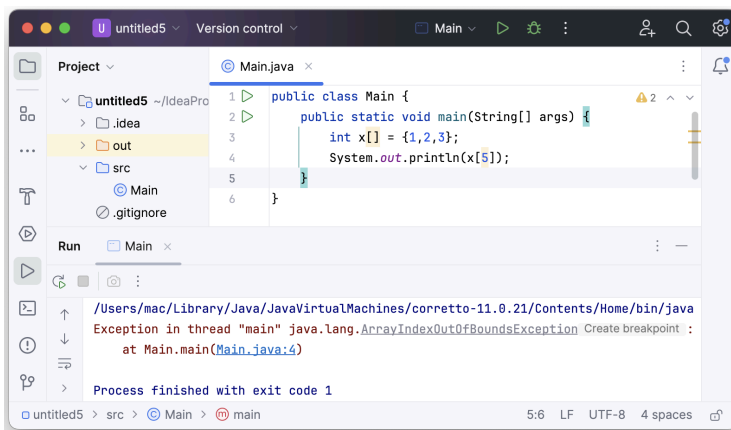


Figure 4-2. The exception generated is displayed in the Terminal window.

Exceptions in Java are represented as objects of type `Exception`. For example, in the case above, the exception is an object of type **`IndexOutOfBoundsException`**. These exceptions are subclasses of **`Exception`**. (Inherited from `Exception`). Different subclasses represent different types of exceptions. There are many types of exceptions. For example, a `NumberFormatException` can occur when trying to convert a string to a number. For example:

```
public static void main(String[] args) {
    int a = Integer.parseInt("123.33");
}
```

```
}
```

Such transformations are performed by methods such as **Integer.parseInt** or **Double.parseDouble**. Consider the method call `Integer.parseInt(str)` where `str` is a variable of type `String`. If the value of `str` is the string “42”, the method call will correctly convert the string to `int 42`. However, if the value of `str` is “forty two”, the method call will fail because “forty two” is not a valid string representation of an `int` value. In this case, a `NumberFormatException` exception is thrown. If nothing is done to handle the exception, the program will close immediately.

4.7.1. Exception Handling

When an exception occurs, we say that the exception is “thrown”. For example, we say that `Integer.parseInt(str)` throws a `NumberFormatException` exception **when** the value of `str` is invalid. When an exception is thrown, it is possible to “catch” the exception and prevent it from crashing the program. This is done with the **try ... catch statement**. In simple form, the syntax for the **try ... catch statement** could be:

```
public static void main(String[] args) {  
    try {  
        //code block-1  
    } catch (exception-class-name variable-name ){  
        //code block-2  
    }  
}
```

In this statement, **exception-class-name** is the name of the exception class, which could be `NumberFormatException`, `IllegalArgumentException`, or some other exception class. When the computer executes this `try..catch` statement, it executes **block-1 in the try section**. If no exception occurs during the execution of **block-1**, the computer simply skips **block-2** and continues with the rest of the program. However, if an exception of type **exception-class-name** occurs during the execution of **block-1**, the computer will immediately jump from the point where the exception occurred to the catch section and execute **block-2**, skipping any remaining statements in **block-1**.

During the execution of **block-2**, **variable-name** can be used to represent the exception object. This variable will contain information about the type and cause of the exception and can be printed to the screen if needed. After the catch is complete, the computer continues with the rest of the program; the exception has been caught and handled and does not crash the program.

Note that curly braces { and } are part of the syntax of the try..catch statement. They are required even when there is only **one statement** between the curly braces. This is different from other statements, where the curly braces around a single statement are optional.

Consider another example: suppose a program finds the average of a series of real numbers entered by the user and asks the user to signal the end of the series by entering a blank line. The user might enter an incorrect value, causing an exception, so a try..catch block is needed to avoid program failure.

```
public static void main ( String [] args) {
    String str; // User input.
    double number; // Input is converted to a number.
    double total; // Sum of all entered numbers.
    double avg ; // Average value of the numbers.
    int count; // The number of numbers entered.
    total = 0 ;
    count = 0 ;
    System.out.println ( " Enter your number, press back to
finish." ) ;
    while ( true ) {
str = new Scanner( System . in ).next();
        if (str.equals( "" ) ) {
            break ; // Exit the loop, because the input line is
empty.
        }
        try {
number = Double.parseDouble ( str); // If an exception occurs
here, the next two lines will be skipped!
            total = total + number;
```

```

count = count + 1 ;
} catch ( NumberFormatException e) {
    System.out.println ( "Invalid number! Please try
again. " ) ;
}
}

avg = total / count;
System.out.printf ( " The average value of %d numbers is
%1.6g%n" , count, avg );
}

```

4.7.2. Exception processing line

A program will include the participation of many different methods, which can call each other, and exceptions can occur at any point in these methods. For example:

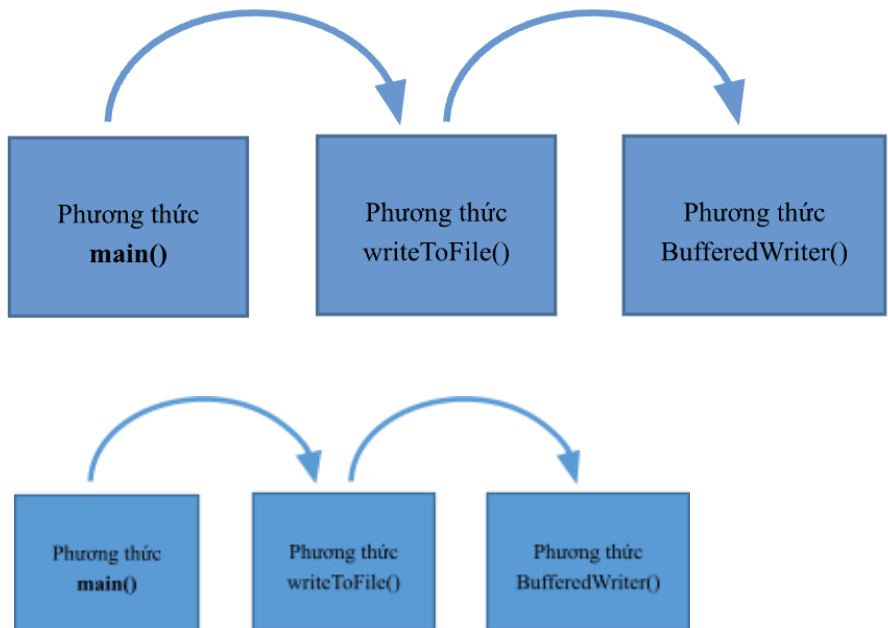


Figure 4-3. Illustration of throwing and catching exceptions.

For example, the **main()** method calls the **writeToFile()** method, which in turn calls input/output methods like **FileWriter()** or **write()**. These

methods perform file opening and writing and may throw an exception, so `writeToFile()` must have a try...catch block as follows:

```
public static void writeToFile () {
    try {
        BufferedWriter bw = new BufferedWriter( new FileWriter(
            "myFile.txt" ));
        bw.write ( "Test" );
        bw.close ();
    } catch ( IOException ex) {
        ex.printStackTrace();
    }
}
public static void main ( String [] args) {
    writeToFile();
}
```

However, if the `writeToFile()` method does not want to handle that exception, it can declare that it throws the exception so that the calling method handles it using the **`throws`** keyword. as follows:

```
public static void writeToFile () throws IOException {
    BufferedWriter bw = new BufferedWriter( new FileWriter(
        "myFile.txt" ));
    bw.write ( "Test" );
    bw.close ();
}

public static void main ( String [] args) {
    try {
        writeToFile();
    } catch ( IOException ex) {
        ex.printStackTrace();
    }
}
```

In the above statement, when an exception occurs, the `printStackTrace()` method of the `ex` object is called within the `catch` block . **This** method will print the **`Stack Trace`** , which is a detailed list

showing the order in which the methods called that caused the exception. For example, 'Stack Trace' appears when the file cannot be opened for writing:

```
java.io.FileNotFoundException: C:/myFile.txt (No such file or
directory)
    at java.base/java.io.FileOutputStream.open0(Native
Method)
    at
java.base/java.io.FileOutputStream.open(FileOutputStream.java
:291)
    at
java.base/java.io.FileOutputStream.<init>(FileOutputStream.ja
va:234)
    at
java.base/java.io.FileOutputStream.<init>(FileOutputStream.ja
va:123)
    at
java.base/java.io.FileWriter.<init>(FileWriter.java:66)
    at Program.writeToFile(Program.java:8)
    at Program.main(Program.java:15)
```

Reading and understanding "Stack Trace" is one of the most important programming skills. For example, in the result above, the exception arises from the main() method calling the writeToFile method (on line 15 of Program.java), the writeToFile() method calling the FileWriter() method (on line 8 of Program.java), the FileWriter() method then calling other methods, and the exception occurs when the open() method fails to open the file. Of course, only the Program.java file is created and programmed by the programmer, so only the lines of code where the exception occurs in this file need to be considered. "Stack Trace" is truly a useful tool for developers to know where to fix errors.

4.7.3. Using finally

Besides using the **try ... catch syntax**, programmers can also use the **try ... catch ... finally syntax**. The syntax is as follows:

```
try {  
  
}  
catch (Exception e) {  
  
}  
finally {  
  
}
```

finally clause declares a block of statements that will be executed as the last step in the execution of the statements in the **try block** , regardless of whether an exception occurs. The statements in the **finally block** occur even if a new exception appears in the **catch block** . This block of code is often used to perform necessary cleanup that should not be skipped under any circumstances, such as closing connections to accessed resources. For example:

```
try {  
// Establish a network connection, perform input/output  
operations  
}  
catch (IOException e ) {  
  
}  
finally {  
//Check if the connection is still open, then close it.  
}
```

In the example above, **finally** ensures that the network connection will definitely be closed, regardless of whether an error occurs or not.

4.7.4. Exception Hierarchy

All exception objects must belong to a subclass of **java.lang.Throwable** . These subclasses are organized in a rather complex hierarchy that shows the relationships between different exception types. **Throwable** has two direct subclasses, **Error** and **Exception**. These two subclasses, in turn, have many other subclasses. Additionally, programmers

can create new exception classes to represent new exception types not included in the standard exception list.

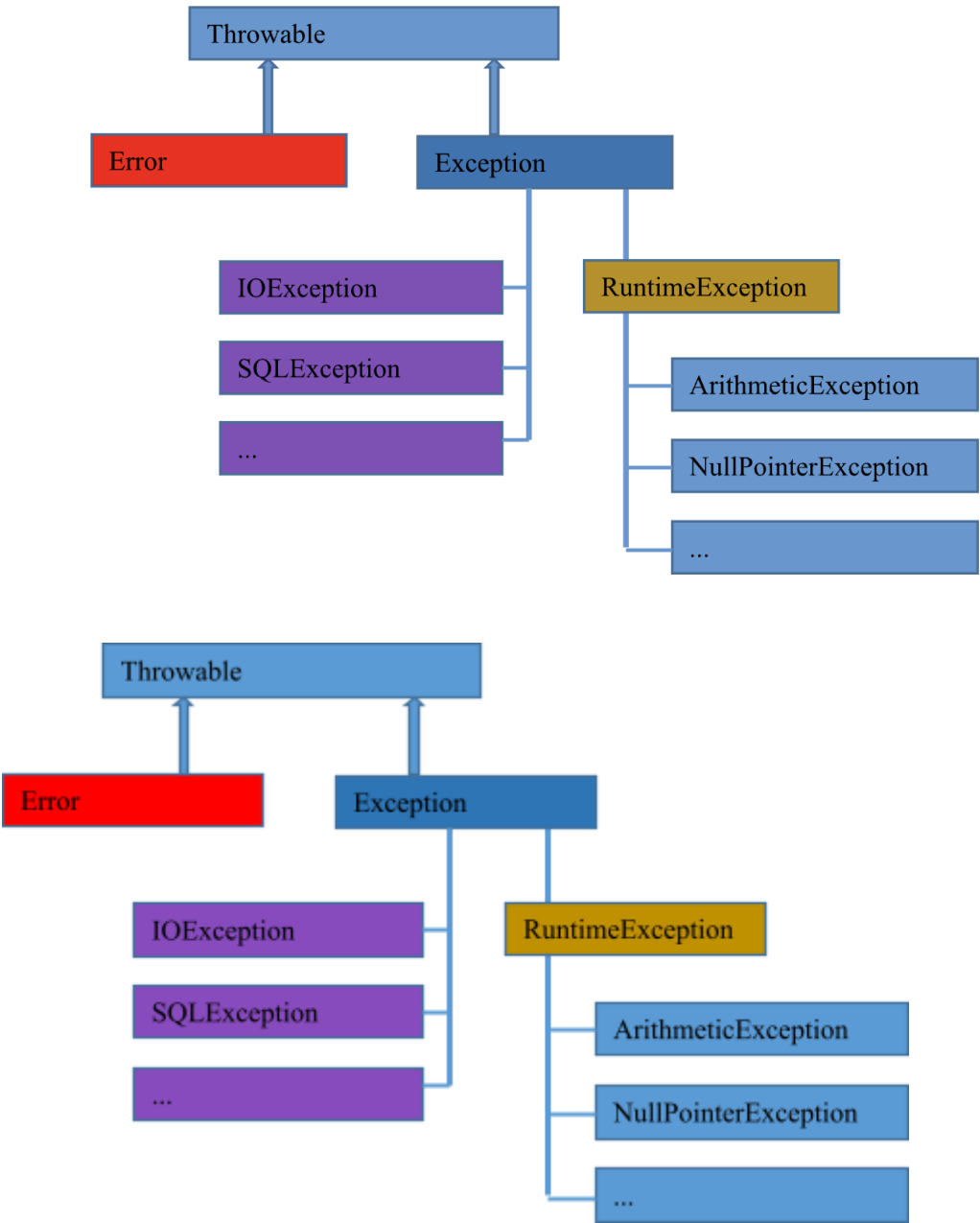


Figure 4-4. Exception class hierarchy diagram

Most subclasses of the **Error** class represent **serious errors** in the

Java Virtual Machine (JVM), such as errors related to corrupted public libraries or memory issues. Generally, it's not advisable to try to catch and handle such errors. For example, `java.lang.OutOfMemoryError` is an error that occurs when the Java Virtual Machine cannot allocate memory for the program and the garbage collector cannot clean up the memory.

On the other hand, subclasses of the **Exception class** represent exceptions that are used for catching. These are "errors" that are not handled in the program and that the programmer has "accidentally" or "intentionally" ignored. The Exception class has a **RuntimeException class**, which represents exceptions that occur when the program runs. Exceptions such as **NullPointerException** or **ArithmeticException** are subclasses of this **RuntimeException class**.

All of these exceptions inherit from **Throwable**. These classes all have important and frequently used methods such as:

- **getMessage()** is a method that returns a string describing the exception.
- **toString()** returns a string containing the name of the class where the exception occurred.
- **printStackTrace()** returns a "stack trace" containing a list showing the detailed order in which the methods called that caused the exception to occur.

Consider another example:

```
double x = arr [ 1 ] / arr [ 2 ];  
System . out .println( "x = " + x );
```

Here, the computer will attempt to execute the "try" block of code. If no exceptions occur during the execution of this block, the "catch" part of the statement will be skipped. However, some situations may arise, such as:

Case 1, the array `arr` only has two elements, so accessing `arr[2]` is an exception. The program will throw an exception called **ArrayIndexOutOfBoundsException**. To ensure that the exception does not occur, you can use a **try...catch block** as follows:

```
try {
    double x = arr [ 1 ]/ arr [ 2 ];
    student.score = x;
}
catch (ArrayIndexOutOfBoundsException ex) {
    System.out.println ( " Array element does not exist");
}
```

However, this does not guarantee that the program is free of exceptions.

Case 2: When the element arr[2] exists in the array but has a value of 0, while arr[1] has a non-zero value, the operation arr [1]/ arr [2] performs a division by zero. This causes an exception of type **ArithmeticException** to appear. The program stops because this exception has not been handled. To handle both types of exceptions, the program needs to be modified as follows:

```
try {
    double x = arr [ 1 ]/ arr [ 2 ];
    student.score = x;
} catch (ArithmeticException ex) {
    System.out.println ( "The operation could not be performed");
} catch (ArrayIndexOutOfBoundsException ex) {
    System.out.println ( " Array element does not exist");
}
```

However, this doesn't guarantee that the program is exception-free. For example, the student object might not be initialized, preventing the statement `student.score = x;` from executing, resulting in a **NullPointerException**. Therefore, the program needs to catch additional **NullPointerExceptions**. While it's now **fairly certain that** all exceptions are handled, some exceptions might be missed. To ensure all exceptions are caught, you can catch exceptions of types such as **RuntimeException**, **Exception**, and others. even It is **Throwable**. For example:

```
try {
```

```

    double x = arr[ 1 ] / arr[ 2 ];
    student.score = x ;
} catch ( ArithmeticException ex) {
    System.out.println ( "The operation could not be
performed" ) ;
} catch ( ArrayIndexOutOfBoundsException ex) {
    System.out.println ( " Array element does not exist" ) ;
} catch ( NullPointerException ex) {
System.out.println ( "Empty object" ) ;
} catch ( RuntimeException ex) {
System.out.println ( "An error occurred" ) ;
}

```

With the code written as above, the system will catch all RuntimeException errors. If necessary, it can also catch more general exceptions such as Exception. However, it is important to note that general exceptions should be placed at the bottom, while exceptions that inherit from general exceptions should be placed at the top. This means that handling exceptions as follows is syntactically incorrect:

```

try {
    double x = arr[ 1 ] / arr[ 2 ];
    student.score = x;
} catch ( Exception ex) {
    System.out.println ( "Error" ) ;
} catch ( ArithmeticException ex) {
    System.out.println ( "The operation could not be performed"
) ;
} catch ( ArrayIndexOutOfBoundsException ex) {
    System.out.println ( " Array element does not exist" ) ;
} catch ( NullPointerException ex) {
    System.out.println ( "Empty object" ) ;
}

```

Clearly, exceptions like **ArithmeticException** , **ArrayIndexOutOfBoundsException**, or **NullPointerException** are all subclasses (children) of the Exception class, so catching exceptions of type Exception should be written as follows:

```

try {
    double x = arr[ 1 ] / arr[ 2 ];
    student.score = x;
} catch ( ArithmeticException ex) {
    System.out.println ( "The operation could not be performed"
) ;
} catch ( ArrayIndexOutOfBoundsException ex) {
    System.out.println ( " Array element does not exist" ) ;
} catch ( NullPointerException ex) {
    System.out.println ( "Empty object" ) ;
} catch ( Exception ex) {
    System.out.println ( "Error" ) ;
}

```

4.7.5. Throwing exceptions and creating exception classes

Keyword **throw** It is used to throw an exception from a method or any block of code. For example:

```

throw new ArithmeticException("Mathematical Exception");

```

Here, a "Mathematics" type exception is created and thrown at any point in the program. The exceptions thrown are usually user-defined exceptions. These exceptions must then be objects created by classes that inherit from existing exception classes or from the Throwable class. For example, create an exception class as follows:

```

public class MyException extends RuntimeException {
    String message ;
    MyException ( String message){
        this.message = message ;
    }

    @Override
    public String getMessage() {
        return message ;
    }
}

```

To throw the above exception, write the following code:

```
throw new MyException( "Custom exception" );
```

Creating an exception class, inheriting from it, and creating objects from a user-defined class will be explored in more detail in the object-oriented programming chapter.

4.7.6. Handling mandatory exceptions

In the previous example of reading a file:

```
public static void writeToFile () throws IOException {
    BufferedWriter bw = new BufferedWriter( new FileWriter(
"myFile.txt" ));
    bw.write ( "Test" );
    bw.close ();
}

public static void main ( String [] args) {
    try {
        writeToFile();
    } catch ( IOException ex) {
        ex.printStackTrace();
    }
}
```

It's clear that in this case, exception handling is mandatory. Failure to do so will result in a syntax error and will be reported by the compiler. Mandatory exceptions are usually those related to input/output operations. They are also called "check exceptions"—some methods always declare that they can "throw" these exceptions. Therefore, methods calling these methods are required to handle the exception. That is, the method must place a statement within the **try block** that includes a clause to handle the exception; in this case, the exception is handled in the subroutine. Another way is to declare that the submethod can throw exceptions. This is done by adding the " **throws** " clause to the method header, which warns any caller about the possibility of an exception being thrown when the method is executed. Thus, the calling method will be forced to handle the exception in

the try statement or declare the exception in the throws clause in its own header.

4.8. JAVA DOCS

When writing methods, the method description should be included in a comment block.

```
/**
```

```
*/
```

For example:

```
/**
```

```
 * Methods for calculating the area of a rectangle
```

```
 * @param width Width
```

```
 * @param height Height
```

```
 * @return Area
```

```
 * @exception IllegalArgumentException occurs when the size is  
negative
```

```
*/
```

```
public static double areaOfRectangle ( double height, double  
width) {
```

```
    if (width < 0 || height < 0 )
```

```
        throw new IllegalArgumentException( "Sides must have  
positive length." );
```

```
    double area ;
```

```
    area = width * height;
```

```
    welcome area ;
```

```
}
```

To generate documentation describing the method, you can use the following command:

```
javadoc Main.java
```

Note that IntelliJ IDEA supports adding JavaDocs and generating JavaDocs, which is very convenient.

PRACTICE

Lesson 1. Finding the divisors of a number

Write a method to calculate and print all the divisors of a given positive integer. The integer will be a parameter of the method. In this case, the method could be declared as:

```
static void printDivisors( int N ) {  
    int D;  
    System.out.println("The divisors of " + N + " are the  
numbers:");  
    for ( D = 1; D <= N; D++ ) {  
        if ( N % D == 0 )  
            System.out.println(D);  
    }  
}
```

Lesson 2. Number Guessing Game

Write a program that plays a guessing game with the user. The computer will choose a random number from 1 to 100, and the user will try to guess it. The computer tells the user whether the guess is high, low, or correct. If the user guesses correctly six or fewer times, the user wins the game. After each game, the user has the option to continue with another game.

```
public class Main {  
  
    public static void main(String[] args) {  
        System.out.println ( "Select a number from 1 to 100" )  
;  
        boolean playAgain;  
        by {  
            playGame ();  
            System.out.print ( "Do you want to play again?" ) ;  
            playAgain = new Scanner(System.in).nextBoolean();  
        } while (playAgain);  
        System.out.println ( "Goodbye" ) ;  
    }  
}
```

```

}

static void playGame() {
    int computersNumber;
    int usersGuess;
    int guessCount;
    computersNumber = ( int ) ( 100 * Math. random () ) + 1
;

    guessCount = 0 ;
    System.out.println ( );
    System.out.print ( "What number do you guess?" ) ;
    while ( true ) {
        usersGuess = new Scanner(System. in ).nextInt();
        expectCount++;
        if (usersGuess == computersNumber) {
            System.out.println ( "You guessed correctly,
the number is: " + computersNumber);
            Break ;
        }
        if (guessCount == 6 ) {
            System.out.println ( "You lost, the number is "
+ computersNumber);
            Break ;
        }
        if (usersGuess < computersNumber)
            System.out.print ( "The number is too low,
guess again: " ) ;
        else if (usersGuess > computersNumber)
            System.out.print ( "The number is too high,
guess again: " ) ;
    }
}
}

```

Lesson 3. Method Overlapping

Write a method called printChar() that prints a character.

```
private static void printChar(char ch) {  
    System.out.print( ch );  
}
```

Write a method with the same name as the one above that prints a line of text containing N copies of the character 'ch'. The method must have one parameter 'ch' of type char and one parameter 'N' of type int.

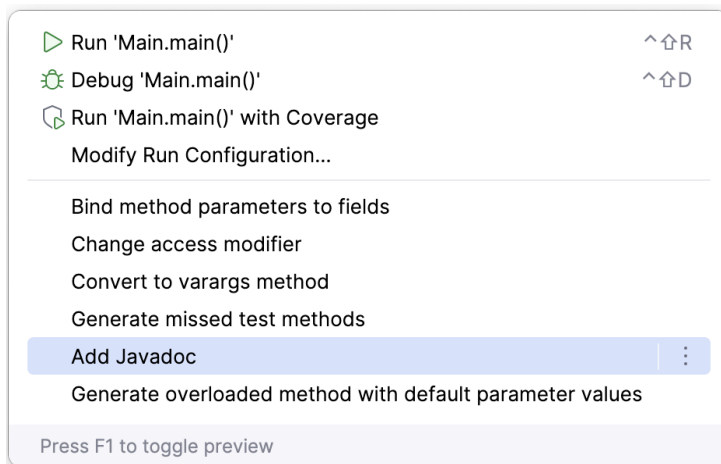
Write a method that takes a string as a parameter. For each character in the string, it should print a line showing 25 copies of that character.

```
private static void printRowsFromString(String str) {  
    for (int i = 0; i < str.length(); i++ ) {  
        printChar(str.charAt(i), 25);  
    }  
}
```

Write a main() method to call and test the above method.

Lesson 4. Creating a JavaDoc

Using IntelliJ IDEA, create a project, declare some methods, and then write JavaDoc for the project by selecting the method and then choosing the Add JavaDoc option.



To create a JavaDoc for a project, select Tools - JavaDoc, then enter the information to generate the JavaDocs folder.

Generate Javadoc

JavaDoc Scope

☒ Whole project
☐ File './src/Teacher.java'
☐ Custom scope:

JavaDoc Options

☐ Include JDK and library sources in -sourcepath
☐ Link to JDK documentation (use -link option)

Output directory:

Visibility level:

☒ Generate hierarchy tree
☐ @use
☒ Generate navigation bar
☐ @author
☒ Generate index
☐ @version
☒ Separate index per letter
☒ @deprecated
☒ Deprecatd list

Locale:

Command line arguments:

Maximum heap size: megabytes

☒ Open generated documentation in browser

?

Cancel

Generate

After clicking Generate, check the generated result.

QUIZ

Which of the following is NOT a rule of a "black box"?	
The black box interface should be simple, clear, and easy to understand.	
The black box needs to conceal all the information and data inside to ensure the security of the program.	
To use a black box, you don't need to know anything about how it works; all you need to know is its interface.	
The person deploying the black box doesn't need to know too much about the larger systems the box will be used in.	

The static keyword is used for:	
Create a static block of code	

Create a static class	
Create a static object	
Create a static method	

Which statement is accurate regarding static variables?	
Static variables must be called through an object.	
Static variables can be accessed by static methods.	
Static variables are created whenever an object is in a static state.	
Static variables cannot have their values changed and are always declared with the keyword 'final'.	

The keyword 'catch' is used to:	
Capture an exception	
Command block navigation	
Shooting exceptions	
Capture an address of the error.	

The keyword finally is used to:	
End the function and return the value.	
End a loop	
Create a block of code that will always be executed when an exception occurs.	
End of a processing sequence	

To throw an exception, use the keyword:	
throws	
catch	
click	
throw	

The method overloading is:	
The act of repeatedly calling methods to perform a specific task.	

Overlapping methods	
Re-declaring a method that already exists in the parent class.	
Declaring methods with the same name but different input parameters.	

A dynamic parameter is a parameter:	
Multiple declarations can be made within a single method.	
Flexible in size	
The style can be flexible.	
It is always declared at the end of the list of parameters passed in.	

Command-line parameters are the following parameters:	
Parameters of the main function	
The parameter passed terminates the program.	
It has the form of an array of integers.	
All of the above options are incorrect.	

Java documentation is generated by the command:	
javadoc	
Java Document	
javahelp	
javadocs	

What is the output of the following code snippet ?	
<pre> public class Program { public static void main (String [] args) { System . out .println(stars (10)); } public String stars (int count) { String res = "" ; for (int i = 0 ; i < count; i ++) { res = res + "*" ; </pre>	

<pre>} return res ; } }</pre>		
Show 9 asterisks *		
Show 10 asterisks *		
Compilation error		
Error while running		

PRACTICE EXERCISES

Lesson 1. Stacking static methods

Write a MathUtils class with the following calculation methods:

- max: takes two integers as input and returns the larger of the two.
- max: takes two real numbers as input and returns the larger of the two numbers.
- max: takes an array of integers as input and returns the largest number in that array.
- max: takes an array of real numbers as input and returns the largest number in that array.

The MathUtils class contains a static main method, in which:

- Create a MathUtils object.
- Call the max method with two integer parameters.
- Call the max method with two real numbers as parameters.
- Create an array of integers and call the max method with this array.
- Create an array of real numbers and call the max method with this array.

Lesson 2. Dynamic Parameters

Write a method that uses dynamic parameters to calculate the sum of any given list of numbers.

Lesson 3. Exception Handling

Write a method to perform the division of two integers. If the second number is 0, the method should throw an exception with the message "Divide by zero".

Lesson 4. JavaDocs Practice

Create a class that includes the following methods:

- Calculate the area of the rectangle.
- Calculate the area of the circle.
- Calculate the area of the triangle.
- Calculate the area of the trapezoid.

Request:

- For each method, complete the instructions.
- Write general instructions for the class.
- Create Javadoc for the Project.

REFERENCES

- [1] Core Java: Fundamentals (2021) , Cay Horstmann (Oracle Press Java).
- [2] Head First Java: A Brain-Friendly Guide (2022), Kathy Sierra, O'Reilly Media.
- [3] Java OOP Done Right: Create object oriented code you can be proud of with modern Java Paperback (2019), Mr Alan Mellor, Mellor Books.
- [4] Murach's Java Programming (5th Edition) (2017), Joe Murach, Mike Murach & Associates.
- [5]. Java for Absolute Beginners Learn to Program the Fundamentals the Java 9+ Way.
- [6]. Modern Java Recipes: Simple Solutions to Difficult Problems in Java 8 and 9 (2017), by Ken Kousen, O'Reilly Media.
- [7] Effective Java (2018), Joshua Bloch, Addison-Wesley Professional.